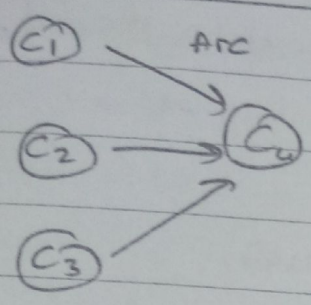
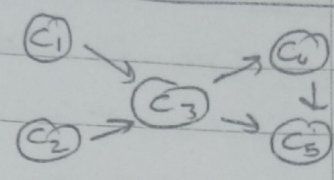




C_1, C_2, C_3, C_4
 C_1, C_2, C_3, C_4



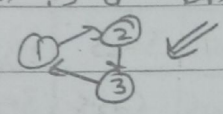
C_1, C_2, C_3, C_4, C_5
 C_2, C_1, C_3, C_4, C_5

Appointment	time
	8
	-30-
	10
	-30-

⇒ When does TS not have a feasible solution?

→ Directed cycle

→ TS has a feasible solution iff the graph is a DAG (Directed Acyclic Graph)



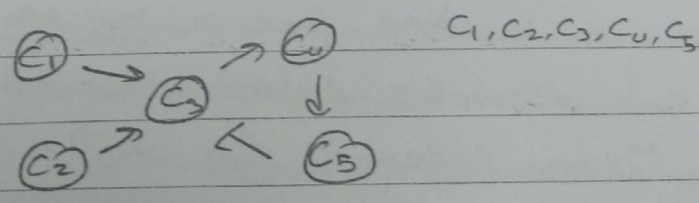
Algorithm - 1 :

⇒ Perform a DFS traversal & note the order in which vertices become dead ends.

(Because if it is a dead end, no other vertex can follow it.) C_1, C_2, C_3, C_4, C_5

Algorithm - 2 :

→ Repeatedly identify in the remaining graph a source (a vertex with no incoming edges), note it down and delete it along with all edges outgoing from it.



⇒ Bu hastadon 1 sonu gelicek

ALGORITHMS FOR GENERATING COMBINATORIAL OBJECTS

“Combinatorics”

Permutation $\{a_1, a_2, a_3, a_4\}$
Combination

⇒ Our goal is generating them, $\{a_1, a_2, a_3, a_4\}$
not counting them.

time	Appointment	Notes
------	-------------	-------

Decrease-by-1 Idea: All subsets of $A = \{a_1, a_2, \dots, a_n\}$ can be divided into 2 groups:

- ① Sets that do not have a_n
- ② " " do have a_n

Step							
Step 0	$\emptyset$						
add a_1 → Step 1	$\emptyset$	$\{a_1\}$					
add a_2 → Step 2	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$			
add a_3 → Step 3	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_3\}$	$\{a_1, a_2\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$
Step 4	"	"	"	"	"	"	"

⊗ All 2^n bit strings of length n can be generated in the same way.
 $\{a_1, a_2, a_3\}$

Bit strings: 000 | 100 | 010 | 110
 Subsets: $\emptyset$ | $\{a_1\}$ | $\{a_2\}$ | $\{a_1, a_2\}$

⊗ Is there a "minimal-change algorithm" for generating bit strings so that every one of them differs from its immediate predecessor by only a single bit?

YES!

Such a sequence of bits is called

"Binary Reflected Gray Code" (BRGC)

(used in telegraphy)

BA: 000

001

011

010

110

111

101

100

n=1	S0S	S1S						
n=2	S00S	S01S	S11S	S10S				
n=3	S000S	S001S	S011S	S010S	S110S	S111S	S01S	S100S

Algorithm BRGC(n)

If $n=1$, make list L_1 containing bit strings 0 & 1 in this order
Else

Generate list L_1 of bit strings of size $n-1$
by calling BRGC($n-1$)

Copy list L_1 to L_2 in reverse order

Add 0 in front of each bit string in L_1

Add 1 " " " " " " " " L_2

Append L_2 to L_1 to get list L

return L

Fake Coin Problem

Among n identical looking coins, one is fake. With a balance scale, we can compare any 2 sets of coins.

→ Continue with dividing by 2.

→ Leaving 1 extra coin aside if n is odd.

→ Divide n coins into 2 piles of $\lfloor \frac{n}{2} \rfloor$ coins each

$$w(n) = 0$$

$$w(n) = w(\lfloor \frac{n}{2} \rfloor) + 1 \quad \text{for } n > 1$$

↓
Identical to worst case # of comparisons in binary search.

$$w(n) = \lfloor \log_2 n \rfloor$$

Russian Peasant Multiplication

n & m : Positive integers

$$\text{If } n \text{ is even} \Rightarrow n.m = \frac{n}{2} \times 2m$$

$$\text{If } n \text{ is odd} \Rightarrow n.m = \left(\frac{n-1}{2} \times 2m \right) + m$$

→ This way, we can compute $n.m$ either recursively or iteratively

15

Saturday
June

WEEK 24

Appointment

Notes

$$85 \times 70 = 5950$$

Josephus Problem

⇒ This algo. leads to very fast HW implementations.

n	m
70	85
35	170
17	340 (C + 170)
8	680 (C + 340)
4	1360
2	2720
1	5440
5440 + 170 + 340 = 5950	

Selection Problem

⇒ find the k^{th} smallest element in a list of n numbers.

(Sorting way: $O(n \log n)$) Can we do it in $O(n)$?

16 Sunday

Partitioning a given list around some value p ^{↗ pivot}

① Left part contains elements $\leq p$

② p : pivot element

③ Right part contains

Lomuto Partitioning

A subarray $A[l \dots r]$ ($0 \leq l \leq r \leq n-1$) composed of 3 continuous segments.

① A segment with elements known to be $\leq p$

② " " " " " " $\geq p$

③ " " " " yet to be compared to p

l r
[p < p ≥ p ?]

On each iteration

- Compare the first element in the unknown segment (pointed to by i) by the pivot p
- If $A[i] \geq p$: Increment i to expand the segment with elements $\geq p$ while shrinking the unprocessed elements
- If $A[i] < p$: The segment with elements $< p$ is to be expanded.

This is done by

- ① Incrementing s (the index of the last element in the first segment)
- ② Swapping $A[i]$ and $A[s]$
- ③ Incrementing i to point to the new first element of the shrink unprocessed element.

Pseudocode Lomuto Partition ($A[l..r]$)

```
p ← A[l]
s ← l
for i ← l+1 to r do
    if A[i] < p
        s ← s+1
        swap(A[s], A[i])
end if
end for
swap(A[s], A[l])
return s
```

⇒ How can we take advantage of a list position to find the k^{th} smallest element in it?

Indices start with 0.

s = Split position of the partition.

① If $s = k-1$, pivot p is the k^{th} smallest element

② If $s > k-1$, k^{th} smallest element is the k^{th} smallest of the left part

③ If $s < k-1$, k^{th} smallest element is the $(k-s)^{\text{th}}$ smallest element of the right part.

time	Appointment	Notes
8		Pseudocode for QuickSelect ($A[l..r], k$) // or any other partitioning algorithm.
-30-		$s \leftarrow \text{LomutoPartition}(A[l..r])$
9		if $s = k-1$ return $A[s]$
-30-		else if $s > k-1$
10		QuickSelect ($A[l..s-1], k$)
-30-		else
11		QuickSelect ($A[s+1..r], k-1-s$)
-30-		end

Partitioning an n -element array requires $n-1$ key comparisons.

Best Case: $C_{\text{best}}(n) \in \Theta(n)$
 $n-1$

Worst Case: An extremely unbalanced partition with 1 part empty & the other part with $(n-1)$ elements.

$$C_{\text{worst}}(n) = (n-1) + (n-2) + \dots + 1 = \frac{n \cdot (n-1)}{2} \in \Theta(n^2)$$

DIVIDE & CONQUER (DSC)

① Divide the problem into several subproblems of the same type; ideally of about equal size

② Solve the subproblems (typically recursively)

③ Combine the solutions to the subproblems to get a solution to the original problem.

→ Spend only linear time for the initial division and final recombining.

Typical recurrence relation for DSC:

$$T(n) = a \cdot T(n/b) + f(n)$$

↳ A function that accounts for dividing the instance of size n into a instances of size n/b & combining their solution.

If $a=1$, DSC covers

"decrease-by-a-constant-factor" algorithms.

MERGESORT

Algorithm MergeSort ($A[0, \dots, n-1]$)

if $n > 1$

copy $A[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$ to $B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$

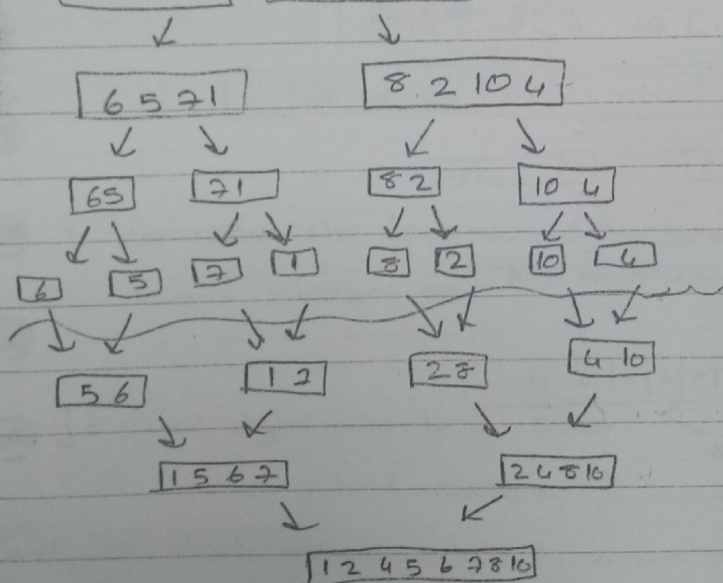
copy $B[\lfloor \frac{n}{2} \rfloor, \dots, n-1]$ to $C[0, \dots, \lceil \frac{n}{2} \rceil - 1]$

MergeSort ($B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$)

MergeSort ($C[0, \dots, \lceil \frac{n}{2} \rceil - 1]$)

Merge (B, C, A)

$\Rightarrow 6, 5, 7, 1, 8, 2, 10, 4$



Recursive call

Combining

$$\Rightarrow C(n) = 2 \cdot C(n/2) + C_{\text{merge}}(n)$$

$\hookrightarrow \in O(n \log n)$

Given 2 arrays,

$\rightarrow$ 2 pointers are initialized to point to the first elements

$\rightarrow$ Compare the elements pointed to, copy the smaller one to the new array & increment the index of the smaller element

$\rightarrow$ Repeat this procedure until one of the 2 given arrays is sorted

Then, the remaining elements are copied to the end of the new array.

ALGORITHM Merge $C[B[0], \dots, p-1], C[p, \dots, q-1], A[0, \dots, p+q-1]$

$i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$

while $C[i] < p$ and $j < q$

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$

$i \leftarrow i+1$

else

$A[k] \leftarrow C[j]$

$j \leftarrow j+1$

end

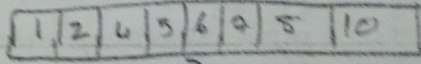
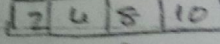
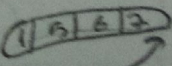
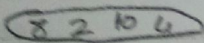
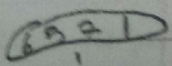
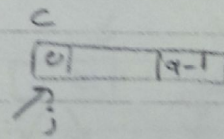
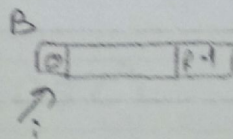
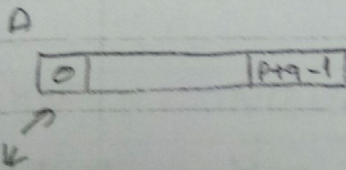
$k \leftarrow k+1$

if $i = p$

copy $C[j, \dots, q-1]$ to $A[k, \dots, p+q-1]$

else

copy $B[i, \dots, p-1]$ to $A[k, \dots, p+q-1]$



Mergesort

→ Advantage: Stability

→ Disadvantage: Requires more memory

QUICKSORT

Unlike mergesort, which divides its input elements according to their position, quicksort divides them according to their value. $\rightarrow$ In quicksort the entire work happens in the division stage.

Partition $\Rightarrow$ Recall Hoare's partition.

ALGORITHM Quicksort ($A[l \dots r]$)

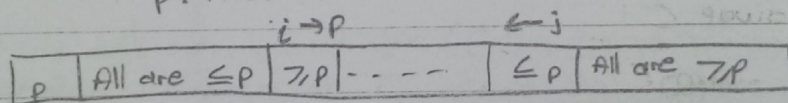
if $l < r$

$s \leftarrow \text{Partition}(A[l \dots r])$ // s is a split position

Quicksort($A[l \dots s-1]$)

Quicksort($A[s+1 \dots r]$)

p : Pivot element



i : Denotes the left-to-right scan and starts with the second element $\Rightarrow$ This scan stops upon encountering the first element $\geq p$.
Pivot = p

j : -- right-to-left scan

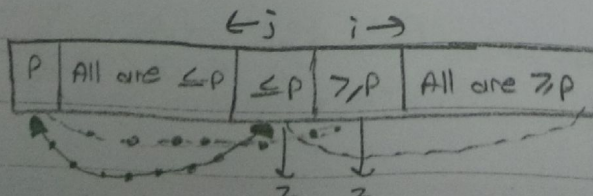
last element $\Rightarrow$...

After both scans stop, 3 situations may arise: ... $\leq \text{pivot}$

Case ① ① If i & j have not crossed (i.e., $i < j$), we simply exchange $A[i]$ and $A[j]$ and resume the scan by incrementing i & decrementing j .

Case ②

② If i and j have crossed over (i.e., $i > j$), we will have partitioned the subarray after exchanging the pivot with $A[j]$.



- Case ⑤ ③ If $i \geq j$ stop while pointing to the same element (i.e., $i=j$), the value they are pointing to must be equal to p .
 This, we already have the subarray partitioned with the split position $s=i=j$.

ALGORITHM Hoare Partition ($A[l \dots r]$)

$p \leftarrow A[l]$ // Pivot

$i \leftarrow l$

$j \leftarrow r+1$

repeat

repeat $i \leftarrow i+1$ until $A[i] \geq p$

repeat $j \leftarrow j-1$ until $A[j] \leq p$

swap ($A[i], A[j]$)

until $i \geq j$

exchange ① Case 2

We are either

(swap ($A[i], A[j]$))

swap ($A[l], A[j]$) $\Rightarrow$ finally, put the pivot in its final place.

return j

② Case 2 or case 3 at this point

Efficiency: The Π of key comparisons made before a partition is achieved:

$$= \begin{cases} n+1 & \text{if the scanning indices cross over.} \\ n & \text{if they coincide.} \end{cases}$$

Best Case:

If all splits happen in the middle of the corresponding subarrays, we have the best case.

$$C_{\text{best}}(n) = 2 \cdot C_{\text{best}}(n/2) + n \quad \text{for } n \geq 1$$

$$C_{\text{best}}(1) = 0$$

Acc. to the Master Theorem
 $C_{\text{best}}(n) \in \mathcal{O}(n \log n)$

Worst Case:

In the worst case, all splits are skewed to the extreme:

- 1 subarray is empty
- The other subarray has size just 1 less than



$$\text{Iteration } \pi 1: \begin{matrix} n+1 \\ \vdots \\ 2 \\ 1 \\ 1 \\ 1 \\ 3 \end{matrix} \left\{ \begin{matrix} \sum_{k=3}^{n+1} k = \frac{(n+1) \cdot (n+2)}{2} - 2 \in O(n^2) \end{matrix} \right.$$

Average Case:

Assuming that the partition split can happen in each position $\leq$ with the same prob. $\frac{1}{n}$, we get the following recurrence relation.

$$C_{avg}(n) = \sum_{s=0}^{n-1} C_{avg}(s) + C_{avg}(n-s-1) + C(n+1)$$

$$C_{avg}(0) = 0$$

$$C_{avg}(1) = 0$$

After the partition,

left & right subarray

will have s &

$n-s-1$ elements

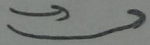
So C_n is too complex

$$C_{avg}(n) \in \dots 1.39 n \cdot \log_2 n$$

$$\in O(n \log n)$$

COUNTING THE NUMBER OF INVERSIONS

5 1 2



$C(5,1)$

$C(5,2)$

Sorted array has 0 inversions

Reverse

$\begin{matrix} n-1 & n-2 & n-3 & n-2 \\ 5 & 4 & 3 & 2 & 1 \end{matrix}$

$$\Rightarrow (n-1) + (n-2) + (n-3) + \dots = \frac{n \cdot (n-1)}{2} \in O(n^2)$$

Meta search engines $\Rightarrow$ Execute the same query on many different search engines & then synthesize the results by comparing their rankings.

CSE 321 HW02

2) a) $f(n) = 3^n + 7^n - 2$

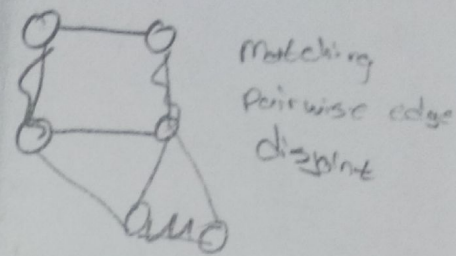
$i=1$ $i \leq n$

$i=2$ $i \leq n$

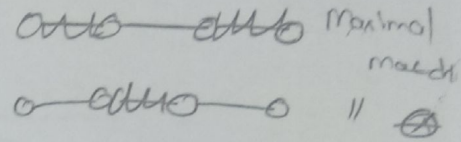
$f(1) = 3^1 + 7^1 - 2 = 8$

$8 \% 8 = 0$

$f(2) = 3^2 + 7^2 - 2 =$



Maximal matching (maximum "1")



u Analik

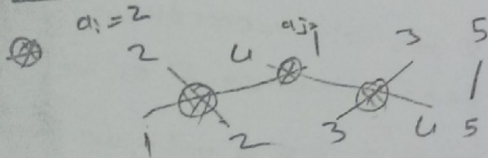
Counting Inversions

2 indices $i < j$ form an inversion if $a_i > a_j$ ("out of order")

2 3 5
3 2 5

Given a sequence of n distinct numbers $a_1, \dots, a_n$, determine the number of inversions in the sequence

Geometric Way:



$n=5$

$\binom{n}{2} = \frac{n \cdot (n-1)}{2} \rightarrow O(n^2)$

lightest
big
 $O(n \log n)$

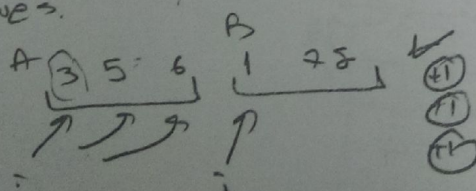
5 4 3 2 1
1 2 3 4 5
Toss almost

$\Rightarrow 5 \ 3 \ 6 \ 1 \ 8 \ 2$

$\rightarrow$ Set $m = \lceil \frac{n}{2} \rceil$ and divide the list into

the 2 pieces $a_1, \dots, a_m$ and $a_{m+1}, \dots, a_n$

$\Rightarrow$ Then count the # of inversions (a_i, a_j) , where the 2 numbers belong to different halves.



$C(3,1)$
 $C(5,1)$
 $C(6,1)$

$\rightarrow$ Make the algorithm recursively sort the numbers in the 2 halves.

$\rightarrow$ After sorting, we have 2 sorted lists A and B.

$\rightarrow$ Produce a single sorted list C from their union, while also counting the number of pairs (a_i, b_j) with $a_i < b_j$ and $a_i > b_j$.

$\rightarrow$ In each step, have a current pointer into each list. Suppose that these pointers point to elements a_i and b_j .

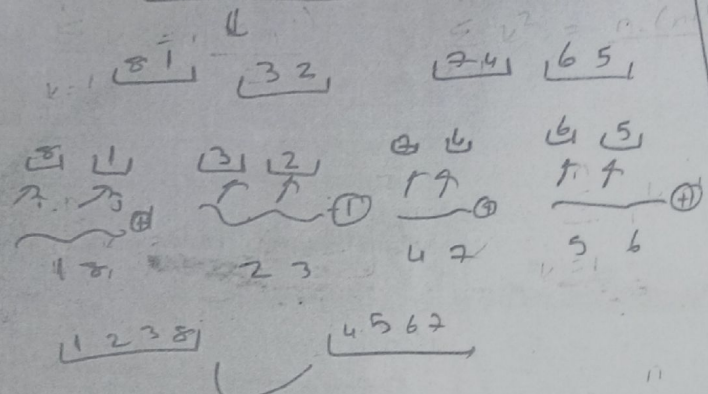
$\rightarrow$ In each step, compare the elements a_i and b_j . remove the smaller from the list and append it to the end of list C.

$\rightarrow$ Every time the element a_i is appended to C, no new inversions are encountered since a_i is smaller than everything left in list B.

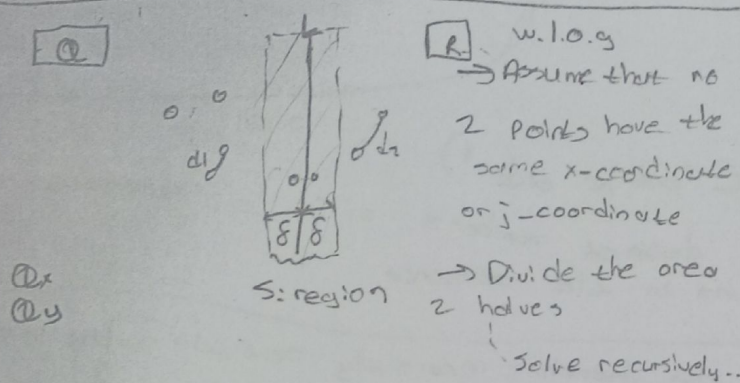
$\rightarrow$ On the other hand, if b_j is appended to list C, then it is smaller than all the remaining items in A. So, we increase our count of the # of inversions by the # of remaining elements in A.

(2)

Combining the Solutions



→ FINDING THE CLOSEST PAIR OF POINTS



→ Combining Step: The distances that have not been considered by either of the recursive calls are precisely those points that occur between a point in the left half and a point in the right half.

On a set of $p \in P$, every recursive call begins with 2 lists:

→ A rise p_i in which all points in p_i have been sorted by increasing x-coordinate.

→ A list P_y' in which all points in P' have been sorted by increasing y-coordinate.

① Δ The set of points in the first $\lceil \frac{n}{2} \rceil$ positions of the list P_n (the "left half")

$R = \begin{matrix} \Delta & \text{"} & \text{"} & \text{"} & \text{"} & \text{"} & \text{"} & \text{final } L\left(\frac{n}{2}\right) \\ \text{"} & \text{"} & \text{"} & \text{"} & \text{"} & \text{"} & \text{"} & R_x(\text{the "right half"}) \end{matrix}$

Q $\triangleq$ Points in Q sorted by increasing x-coordinate.

~~$$a_y = \frac{\Delta}{\Delta t}$$
$$b_x = \frac{\Delta}{\Delta t}$$
$$b_y = \frac{\Delta}{\Delta t}$$~~

$\rightarrow a_0^k$ and a_1^k : Closest pair of points in A

$\rightarrow r_1$ and r_1^* : closest " " " " in R

$$f = \min \{ d(\alpha_0^*, \alpha_1^*)^{\text{distance}}, d(\alpha_0^*, \alpha_1^*) \}$$

→ Are there points $q \in Q$ and $r \in R$ for which
 $d(q, r) < \delta$?

$x^* = \frac{\Delta}{2}$ x-coordinate of the rightmost point in $\mathcal{Q}$

Lemma 1.5 $\exists q \in Q$ and $r \in \mathbb{R}$ for which $d(q, n) < \delta$, then each of $\underline{q}$ and $\underline{r}$ lies within distance δ of L .

Proof: Suppose that such q and r exist $q = (q_x, q_y)$ and $r = (r_x, r_y)$. By definition of x^* , we know that

$$\alpha_1 x \leq x^{\frac{1}{2}} \leq r_1 x$$

Then we have:

Then we have:

$$x^* - q_x \leq r_x - a_x \leq d(q, r) < \delta \quad \text{and}$$

$$r_x - x^k \leq r_x - q_x \leq d(q, r) < \delta$$

So, each of a and r has an x -coordinate within δ of x^* and lies within distance δ of line L .

④ So, if we want to find a close q and r , we can restrict our search to the narrow band consisting only of points in P within δ of d .

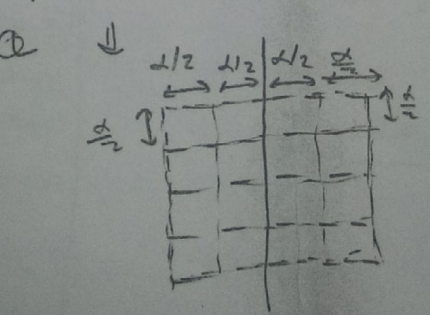
only at points $a \in \mathbb{R}$ and $b \in \mathbb{R}$ for which $d(a, b) < \delta$

(*) There exist $q \in \mathbb{Q}$ and $r \in \mathbb{R}$ for which $d(q, r) < \delta$

iff $\exists s, s' \in S$ for which $d(s, s') < \delta$

lemma: If $s, s' \in S$ have the property that $(s, s') \in \delta$, then s and s' are within 15 positions of each other in the sorted list S_y .

pf. Consider the subset Z of the plane consisting of all points within distance δ of L . Partition Z into boxes as in here.



at prove that each box contains at most 1 point of S .

Suppose that 2 points s lie in the same box. Since all points in the box lie on the same side of d , these 2 points either both belong to Q or both belong to R .

However, any 2 points in the same box are within distance $\frac{\delta r_2}{2} < \delta$ which contradicts our definition of δ as the minimum distance between any pair of points in Q or in R .

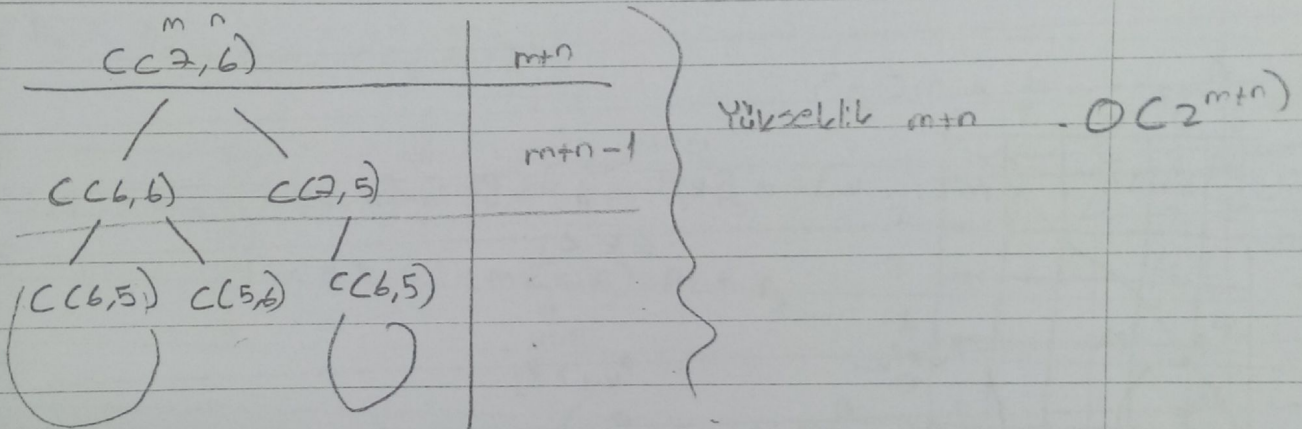
Thus, each box contains at most 1 point of S .

Ex: $X = A B C B D A B = n = 7$
 $Y = B D C A B A B = m = 7$ } Brute force çözüm $O(2^{m \cdot n})$

Recursive çözüm: $C[i, j] = LCS(X[1 \dots i], Y[1 \dots j])$

$$C[i, j] = \begin{cases} C[i-1, j-1] + 1 & X[i] = Y[j] \\ \max(C[i-1, j], C[i, j-1]) & \text{else} \end{cases}$$

$$\left. \begin{array}{l} C[0, 0] = 0 \\ C[k, 0] = 0 \\ C[0, k] = 0 \end{array} \right\} \text{Base-case'ler}$$



Dynamic çözüm:

$X = A B C B D A B$, $Y = B D C A B A$

	B	D	C	A	B	A
B	0	1	0	0	1	1
D	0	1	1	0	1	0
C	0	0	1	1	0	0
A	0	0	0	1	0	1
B	0	1	0	0	1	1
D	0	1	1	0	1	0
C	0	0	1	1	0	0
A	0	0	0	1	0	1
B	0	1	0	0	1	1

$O(m \cdot n)$ 'da çözülür

$\Rightarrow B D A B$

$\Rightarrow B C B A$

time	Appointment	Notes
------	-------------	-------

$$\Rightarrow A \quad B \quad P[A] \quad P[B] = P[\boxed{A \quad B}] \quad Pqr$$

$p \times q \quad q \times r$

$$\begin{matrix} p \times q & q \times r & r \times s \\ A & B & C \end{matrix} = (AB) \cdot C = A \cdot (BC)$$

$10 \times 100 \quad 100 \times 5 \quad 5 \times 50$

$$BC = 100 \times 5 \times 50 = 25,000$$

$$AB = 10 \times 100 \times 5 = 5,000$$

$$10 \times 5 \times 50 = 2,500$$

$$\underline{\quad 2,500 \quad}$$

$$p_0, p_1, p_2, \dots, p_{n-1}, p_n$$

$$A_1, A_2, \dots, A_n$$

18

$$\Rightarrow A_i \cdot A_{i+1} \cdot A_{i+2} \cdot \dots \cdot A_j = M[i, j]$$

20

$$M[i, j] = \min \left\{ M[i, \dots, k] + M[k+1, \dots, j] + p_{i-1} \cdot p_k \cdot p_j \right. \\ \left. : i \leq k < j \right\}$$

6 Sunday

$$\begin{matrix} p_0 \times p_1 & p_{k-1} \times p_k & p_k \times p_{k+1} & p_{k-1} \times p_j \\ (A_1 \cdot A_2 \cdot \dots \cdot A_k) & - & (A_{k+1} \cdot A_{k+2} \cdot \dots \cdot A_j) \\ p_0 \times p_k & & p_k \cdot p_j \end{matrix}$$

$$\rightarrow A_1 \cdot (A_2 \cdot A_3 \cdot A_4 \cdot A_5) + p_{\dots}$$

$$\rightarrow (A_1 \cdot A_2) \cdot (A_3 \cdot A_4 \cdot A_5) + \dots$$

$$\rightarrow (A_1 \cdot A_2 \cdot A_3) \cdot (A_4 \cdot A_5) + \dots$$

$$\rightarrow (A_1 \cdot A_2 \cdot A_3 \cdot A_4) \cdot A_5 + \dots$$

15

$$\text{Example} = A_1 \cdot A_2 \cdot A_3 \cdot A_4 \cdot A_5$$

$$\begin{matrix} 6 \times 10 & 10 \times 3 & 3 \times 12 & 12 \times 20 & 20 \times 2 \\ p_0 & p_1 & p_2 & p_3 & p_4 & p_5 \end{matrix}$$

$$M[i, j] = \min \left(M[i, \dots, k] + M[k+1, \dots, j] + p_{i-1} \cdot p_k \cdot p_j \right) \\ i \leq k < j$$

0

1

	1	2	3	4	5
1	0	120	264	1080	1344
2		0	360	1320	1350
3			0	720	1140
4				0	1650
5					0

$$m[i, i] = 0$$

$$\min \begin{cases} m[1,1] + m[2,3] + p_0 \cdot p_1 \cdot p_3 = 864 \\ m[1,2] + m[3,3] + p_0 \cdot p_2 \cdot p_3 = 264 \end{cases}$$

$$A_1, A_2, A_3 = ((A_1, A_2), A_3)$$

$$\rightarrow A_2, A_3, A_4 = \min \begin{cases} m[2,2] + m[3,4] + p_1 \cdot p_2 \cdot p_4 = 1320 \\ m[2,3] + m[4,4] + p_1 \cdot p_3 \cdot p_4 \end{cases}$$

$$A_2, A_3, A_4 = (A_2, (A_3, A_4))$$

$$m[3,5] = \min \begin{cases} m[3,3] + m[4,5] + p_2 \cdot p_3 \cdot p_5 \\ m[3,4] + m[5,5] + p_2 \cdot p_4 \cdot p_5 \end{cases}$$

	1	2	3	4	5
1		41	2	2	2
2			41	2	2
3				41	4
4					41
5					

Parameter-
table

$$m[1,4] = \min \begin{cases} m[1,1] + m[2,4] + p_0 \cdot p_1 \cdot p_4 = 2120 \\ m[1,2] + m[3,4] + p_0 \cdot p_2 \cdot p_4 = 1080 \\ m[1,3] + m[4,4] + p_0 \cdot p_3 \cdot p_4 \end{cases}$$

$$(A_1, A_2) \cdot (A_3, A_4)$$

$$m[2,5] = \min \begin{cases} m[2,2] + m[3,5] + p_1 \cdot p_2 \cdot p_5 = 1350 \\ m[2,3] + m[4,5] + p_1 \cdot p_3 \cdot p_5 = \\ m[2,4] + m[5,5] + p_1 \cdot p_4 \cdot p_5 \end{cases}$$

$$A_2, A_3, A_4, A_5 = A_2 \cdot (A_3, A_4, A_5)$$

$$m[1,5] = \min \begin{cases} m[1,1] + m[2,5] + p_0 \cdot p_1 \cdot p_5 = 1630 \\ m[1,2] + m[3,5] + p_0 \cdot p_2 \cdot p_5 = 1366 \\ m[1,3] + m[4,5] + p_0 \cdot p_3 \cdot p_5 \\ m[1,4] + m[5,5] + p_0 \cdot p_4 \cdot p_5 \end{cases}$$

$$(A_1, A_2) \cdot ((A_3, A_4), A_5)$$

$3 + 3 = 6$ dyn!

Now suppose that $s, s' \in S$ have the property that $d(s, s') < \delta$ and that they are at least

Assum
tion.

16 positions apart in Sy.

without loops or generality.

Assume w.l.o.g. that s has the smaller y -coordinate.

Then, since there can be most 1 point per

However, any 2 points in $\mathbb{Z}$ separated by at least 3 rows must be a distance

of less $\frac{3.0}{2}$, a contradiction. 

→ $C_{x, 7, 16}$, ($x \leq 15$)

We make 1 pass through S_y and for each $z \in S_y$, we compute its distance to each of the next 15 points in S_y .

This way, we can compute the distance of each pair of points in S that are at distance less than δ from each other.

Weighted Interval Scheduling

⇒ You have a recurse (lecture room, computer)

A request takes the form : Can I reserve the resource starting at time s until time f ?

In this problem,

⑦ resource can be used by at most 1 person/job at a time.

→ Accepted requests should NOT overlap in time.

We have n requests labeled $1, \dots, n$

s_i = start time of request i

f. = Finish " " "

V_i = Value of request i

2 intervals are compatible if they do not overlap.

The goal is to select a subset $S \subseteq \{1, \dots, n\}$ of mutually compatible intervals so as to maximize $\sum_{i \in S} v_i$.

Suppose that the requests are sorted in order of nondecreasing finish time: $f_1 \leq f_2 \leq \dots \leq f_n$.

$\Rightarrow$ We say that a request i comes before a request j if $i < j$.

$p(i) =$ The largest index $i < j$ for an interval j such that intervals i and j are disjoint.

Index

1	$v_1=2$	$p(1)=0$
2	$v_2=6$	$p(2)=0$
3	$v_3=6$	$p(3)=1$
4	$v_4=2$	$p(4)=0$
5	$v_5=2$	$p(5)=3$
6	$v_6=1$	$p(6)=3$

Consider an optimal solution O^* .

Interval n either belongs to O^* or it does not.
(Job)

If $n \in O^* \Rightarrow$ No interval indexed strictly between $p(n)$ & n can belong to O^* [by the definition of $p(n)$]

$\Leftarrow$

Then, O^* must include an optimum solution to the problem consisting of requests $\{1, \dots, p(n)\}$, because otherwise we could replace O^* 's choice of requests from $\{1, \dots, p(n)\}$ with a better one, with no danger of overloading request n .

12

Saturday
October

WEEK 41

time

Appointment

Notes

8

9

10

11

12

13

14

15

16

17

18

19

20

21

If $n \neq 0^*$, then 0^* is simply equal to the optimum solution to the problem consisting of requests $\{1, \dots, n-1\}$.

Let $0^*(j)$ denote the optimum solution to the problem consisting of requests $\{1, \dots, j\}$.

Let $OPT(j)$ = the value of this solution.

(Define $OPT(0) = 0$)

We are seeking $0^* = OPT(n)$

$$OPT(j) = \max \left\{ \underbrace{v_j + OPT(p(j))}_{\text{when } j \in OPT(j)}, OPT(j-1) \right\}$$

Request j can belong to an optimal solution on the set $\{1, \dots, j\}$ if and only if $v_j + OPT(p(j)) \geq OPT(j-1)$

13

Sunday

8

9

10

11

12

13

14

15

16

17

18

19

20

21

ALGORITHM:

Compute $OPT(j)$

if $j = 0$ then

Return 0

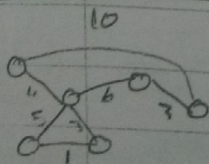
else

Return $\max (v_j + \text{Compute } OPT(p(j)), \text{Compute } OPT(j-1))$

Endif

GREEDY ALGORITHMS

Minimum Spanning tree



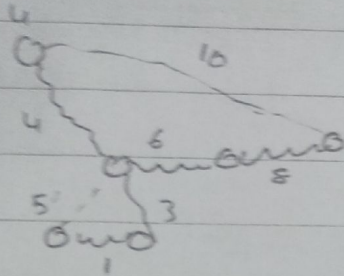
Tree: Connected
&
acyclic

↳ Build up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion

Prim Algo: Start with a root node & greedily grow a tree outwards

Kruskal Algo

Reverse-Delete Algo



Interval Scheduling

which rule?

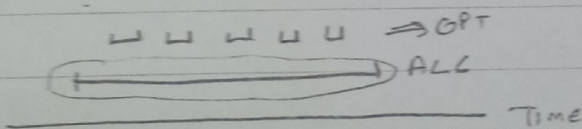
Basic Idea: Use a simple rule to select the first request i_1 .

Reject all requests that are incompatible with i_1 .

Then select the next request i_2 & again reject

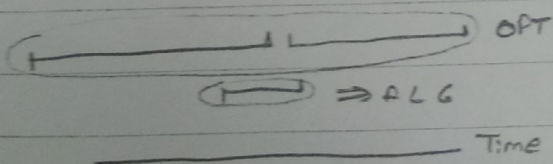
Continue until we run out of request's

Option-1: Select the available request that starts earliest. Not OPT!



Option-2: Select the request that requires the smallest interval of time

(with minimum $f(i) - s(i)$)



Option-3: For each request, count the π of other requests that are not compatible & accept the request with fewest π of conflicts

Ex:

